

Dynamic Programming – Optimization Problems

0/1 Knapsack, Travelling Salesperson Problem (DP version), Optimal Binary Search Tree

Dr SRIRAMYA

Name: P.Bhanu Prakash (192425263)

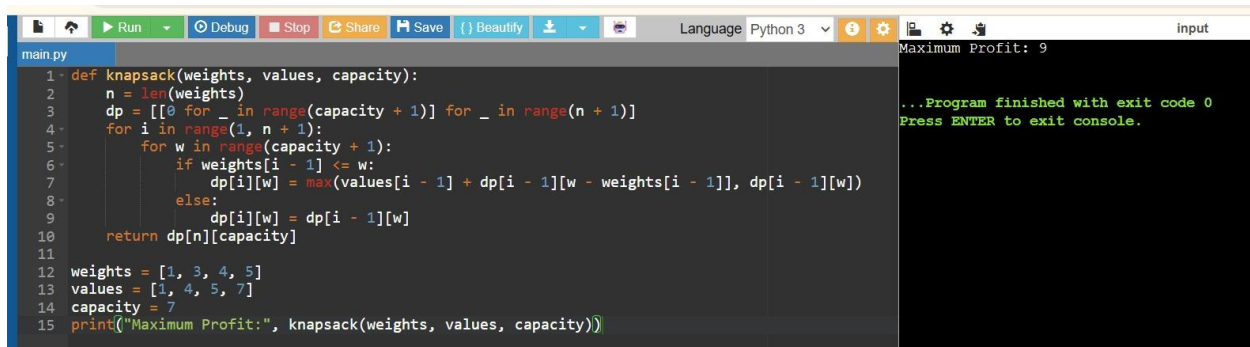
1. 0/1 Knapsack Problem (Dynamic Programming)

```
def knapsack(weights, values, capacity):  
    n = len(weights)  
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]  
    for i in range(1, n + 1):  
        for w in range(capacity + 1):  
            if weights[i - 1] <= w:  
                dp[i][w] = max(  
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],  
                    dp[i - 1][w]  
                )  
            else:  
                dp[i][w] = dp[i - 1][w]  
    return dp[n][capacity]  
  
weights = [1, 3, 4, 5]  
values = [1, 4, 5, 7]  
capacity = 7
```

```
print("Maximum Profit:", knapsack(weights, values, capacity))
```

Output

Maximum Profit: 9



The screenshot shows a Python IDE with a dark theme. The left pane displays the code for the 0/1 knapsack problem, including the knapsack function and the main execution block. The right pane shows the output: 'Maximum Profit: 9'. Below the output, a message indicates the program finished with exit code 0 and prompts the user to press ENTER to exit the console.

```
main.py  
1 def knapsack(weights, values, capacity):  
2     n = len(weights)  
3     dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]  
4     for i in range(1, n + 1):  
5         for w in range(capacity + 1):  
6             if weights[i - 1] <= w:  
7                 dp[i][w] = max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w])  
8             else:  
9                 dp[i][w] = dp[i - 1][w]  
10    return dp[n][capacity]  
11  
12 weights = [1, 3, 4, 5]  
13 values = [1, 4, 5, 7]  
14 capacity = 7  
15 print("Maximum Profit:", knapsack(weights, values, capacity))  
  
Maximum Profit: 9  
...Program finished with exit code 0  
Press ENTER to exit console.
```

2. Travelling Salesperson Problem (Dynamic Programming)

```
from functools import lru_cache
```

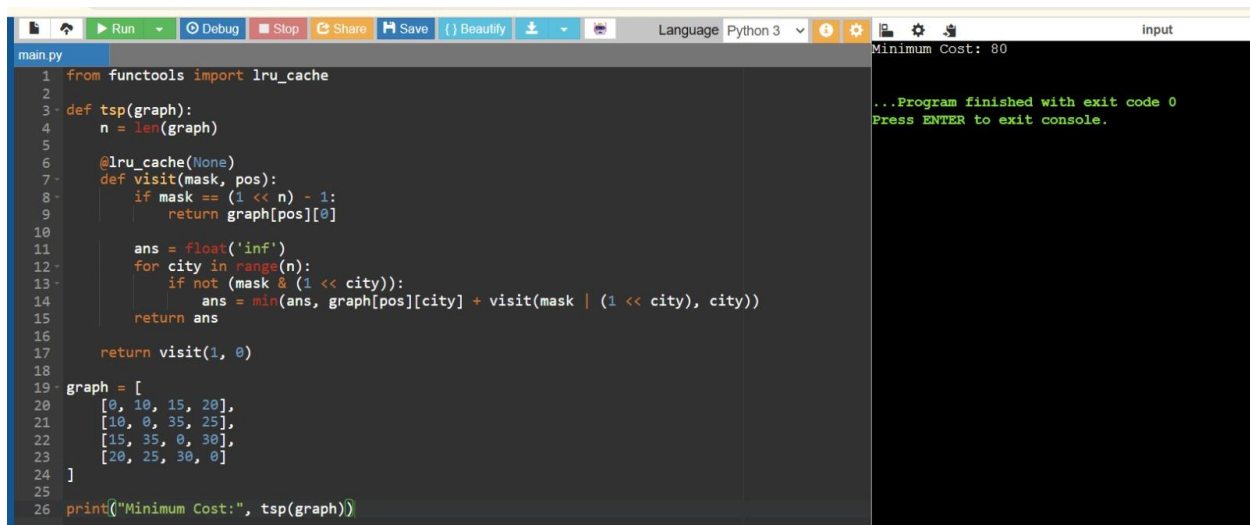
```
def tsp(graph):
    n = len(graph)
    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]
        ans = float('inf')
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    ans,
                    graph[pos][city] +
                    visit(mask | (1 << city), city)
                )
        return ans
    return visit(1, 0)
```

```
graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

print("Minimum Cost:", tsp(graph))
```

Output

Minimum Cost: 80



The screenshot shows a code editor with a dark theme. The left pane displays the Python code for the TSP solution, including the `lru_cache` decorator and the `visit` function. The right pane shows the output of the program, which is "Minimum Cost: 80". Below the output, a message indicates that the program finished with exit code 0 and prompts the user to press ENTER to exit the console.

```
main.py
1 from functools import lru_cache
2
3 def tsp(graph):
4     n = len(graph)
5
6     @lru_cache(None)
7     def visit(mask, pos):
8         if mask == (1 << n) - 1:
9             return graph[pos][0]
10
11         ans = float('inf')
12         for city in range(n):
13             if not (mask & (1 << city)):
14                 ans = min(ans, graph[pos][city] + visit(mask | (1 << city), city))
15         return ans
16
17     return visit(1, 0)
18
19 graph = [
20     [0, 10, 15, 20],
21     [10, 0, 35, 25],
22     [15, 35, 0, 30],
23     [20, 25, 30, 0]
24 ]
25
26 print("Minimum Cost:", tsp(graph))
```

Minimum Cost: 80

...Program finished with exit code 0
Press ENTER to exit console.

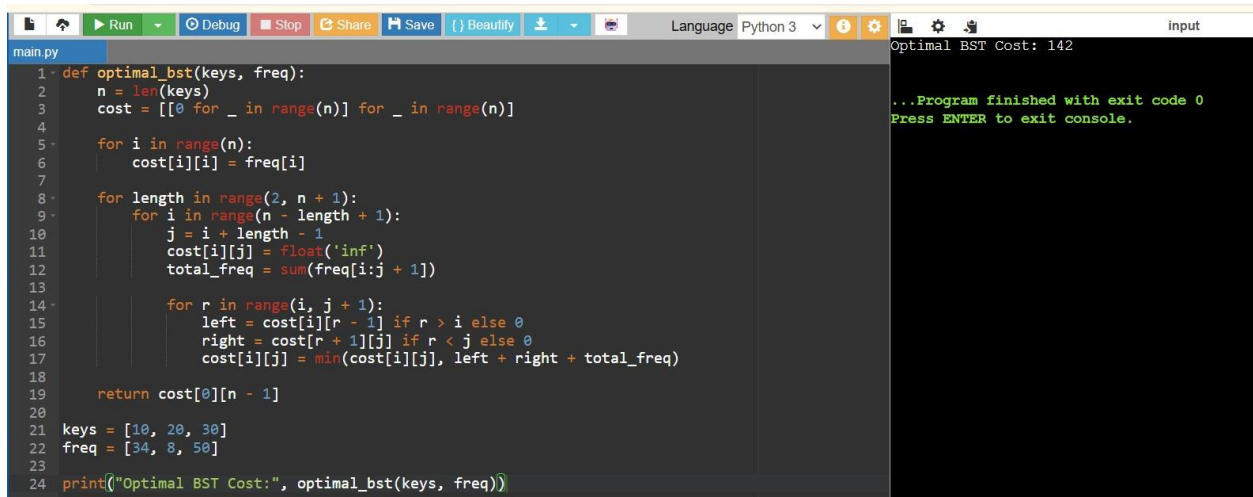
3. Optimal Binary Search Tree (OBST)

```
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float('inf')
            total_freq = sum(freq[i:j + 1])
            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(
                    cost[i][j],
                    left + right + total_freq
                )
    return cost[0][n - 1]

keys = [10, 20, 30]
freq = [34, 8, 50]
print("Optimal BST Cost:", optimal_bst(keys, freq))
```

Output

Optimal BST Cost: 142



```
1- def optimal_bst(keys, freq):
2-     n = len(keys)
3-     cost = [[0 for _ in range(n)] for _ in range(n)]
4-
5-     for i in range(n):
6-         cost[i][i] = freq[i]
7-
8-     for length in range(2, n + 1):
9-         for i in range(n - length + 1):
10-             j = i + length - 1
11-             cost[i][j] = float('inf')
12-             total_freq = sum(freq[i:j + 1])
13-
14-             for r in range(i, j + 1):
15-                 left = cost[i][r - 1] if r > i else 0
16-                 right = cost[r + 1][j] if r < j else 0
17-                 cost[i][j] = min(cost[i][j], left + right + total_freq)
18-
19-     return cost[0][n - 1]
20-
21- keys = [10, 20, 30]
22- freq = [34, 8, 50]
23-
24- print("Optimal BST Cost:", optimal_bst(keys, freq))
```

Optimal BST Cost: 142

...Program finished with exit code 0
Press ENTER to exit console.

4. 0/1 Knapsack Problem

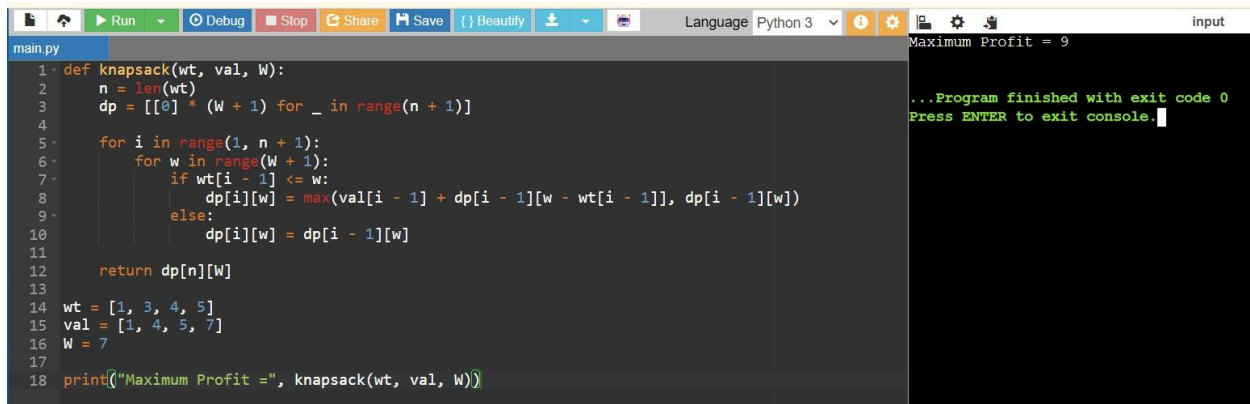
Program

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0]*(W+1) for _ in range(n+1)]
    for i in range(1, n+1):
        for w in range(W+1):
            if wt[i-1] <= w:
                dp[i][w] = max(val[i-1] + dp[i-1][w-wt[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][W]

wt = [1, 3, 4, 5]
val = [1, 4, 5, 7]
W = 7
print("Maximum Profit =", knapsack(wt, val, W))
```

Output

Maximum Profit = 9

A screenshot of a code editor window. The left pane shows a Python script named 'main.py' with the 0/1 Knapsack algorithm implementation. The right pane shows the output of the program, which is 'Maximum Profit = 9'. Below the output, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.' The editor has a dark theme and various toolbars at the top.

5. 0/1 Knapsack Using Memoization

Program

```
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0
    if memo[n][W] != -1:
        return memo[n][W]
    if wt[n-1] <= W:
        memo[n][W] = max(
            val[n-1] + knapsack(W-wt[n-1], wt, val, n-1, memo),
            knapsack(W, wt, val, n-1, memo)
        )
```

```

        else:
            memo[n][W] = knapsack(W, wt, val, n-1, memo)
        return memo[n][W]

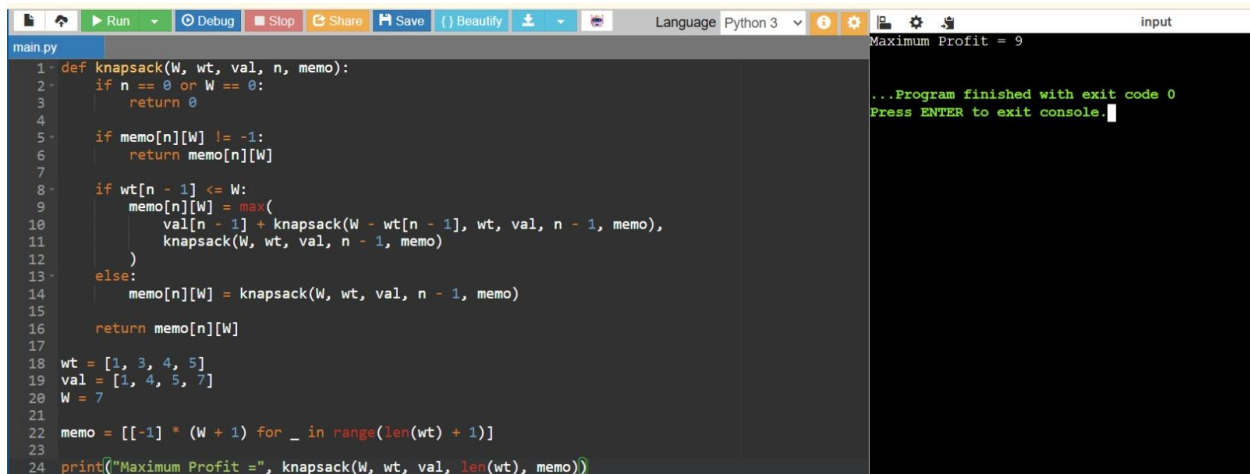
wt = [1,3,4,5]
val = [1,4,5,7]
W = 7

memo = [[-1]*(W+1) for _ in range(len(wt)+1)]
print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))

```

Output

Maximum Profit = 9



The screenshot shows a Python IDE with a dark theme. The left pane displays the code for the knapsack problem, and the right pane shows the output. The code defines a recursive function `knapsack` with memoization, sets up the weight and value arrays, and prints the maximum profit. The output in the right pane shows 'Maximum Profit = 9' and a message indicating the program finished with exit code 0.

```

main.py
1 def knapsack(W, wt, val, n, memo):
2     if n == 0 or W == 0:
3         return 0
4
5     if memo[n][W] != -1:
6         return memo[n][W]
7
8     if wt[n-1] <= W:
9         memo[n][W] = max(
10             val[n-1] + knapsack(W - wt[n-1], wt, val, n-1, memo),
11             knapsack(W, wt, val, n-1, memo)
12         )
13     else:
14         memo[n][W] = knapsack(W, wt, val, n-1, memo)
15
16     return memo[n][W]
17
18 wt = [1, 3, 4, 5]
19 val = [1, 4, 5, 7]
20 W = 7
21
22 memo = [[-1] * (W + 1) for _ in range(len(wt) + 1)]
23
24 print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))

```

Maximum Profit = 9

...Program finished with exit code 0
Press ENTER to exit console.

6. Subset Sum Problem

Program

```

def subset_sum(arr, target):
    n = len(arr)
    dp = [[False]*(target+1) for _ in range(n+1)]
    for i in range(n+1):
        dp[i][0] = True
        for i in range(1, n+1):
            for j in range(1, target+1):
                if arr[i-1] <= j:
                    dp[i][j] = dp[i-1][j] or dp[i-1][j-arr[i-1]]
                else:
                    dp[i][j] = dp[i-1][j]
    return dp[n][target]

print(subset_sum([3,34,4,12,5,2], 9))

```

Output

True

```
main.py
1 def subset_sum(arr, target):
2     n = len(arr)
3     dp = [[False] * (target + 1) for _ in range(n + 1)]
4
5     for i in range(n + 1):
6         dp[i][0] = True
7
8     for i in range(1, n + 1):
9         for j in range(1, target + 1):
10             if arr[i - 1] <= j:
11                 dp[i][j] = dp[i - 1][j] or dp[i - 1][j - arr[i - 1]]
12             else:
13                 dp[i][j] = dp[i - 1][j]
14
15     return dp[n][target]
16
17 print(subset_sum([3, 34, 4, 12, 5, 2], 9))
```

```
True
...Program finished with exit code 0
Press ENTER to exit console.
```

7. Travelling Salesperson Problem (DP Version)

Program

```
from functools import lru_cache
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]
n = len(graph)
@lru_cache(None)
def tsp(mask, pos):
    if mask == (1<<n)-1:
        return graph[pos][0]
    ans = float('inf')
    for city in range(n):
        if not mask & (1<<city):
            ans = min(ans,
                graph[pos][city] +
                tsp(mask | (1<<city), city))
    return ans
print("Minimum Cost =", tsp(1,0))
```

Output

Minimum Cost = 80

```

1 from functools import lru_cache
2
3 graph = [
4     [0, 10, 15, 20],
5     [10, 0, 35, 25],
6     [15, 35, 0, 30],
7     [20, 25, 30, 0]
8 ]
9
10 n = len(graph)
11
12 @lru_cache(None)
13 def tsp(mask, pos):
14     if mask == (1 << n) - 1:
15         return graph[pos][0]
16
17     ans = float('inf')
18
19     for city in range(n):
20         if not (mask & (1 << city)):
21             ans = min(
22                 ans,
23                 graph[pos][city] + tsp(mask | (1 << city), city)
24             )
25
26     return ans
27
28 print("Minimum Cost =", tsp(1, 0))

```

Minimum Cost = 80

...Program finished with exit code 0
Press ENTER to exit console.

8. Travelling Salesperson Using DP Table

Program

```

def tsp_dp(graph):
    n = len(graph)
    dp = [[float('inf')]*n for _ in range(1<<n)]
    dp[1][0] = 0
    for mask in range(1<<n):
        for u in range(n):
            if mask & (1<<u):
                for v in range(n):
                    if not mask & (1<<v):
                        dp[mask | (1<<v)][v] = min(
                            dp[mask | (1<<v)][v],
                            dp[mask][u] + graph[u][v]
                        )
                ans = float('inf')
                for i in range(1,n):
                    ans = min(ans,
                               dp[(1<<n)-1][i] + graph[i][0])
    return ans

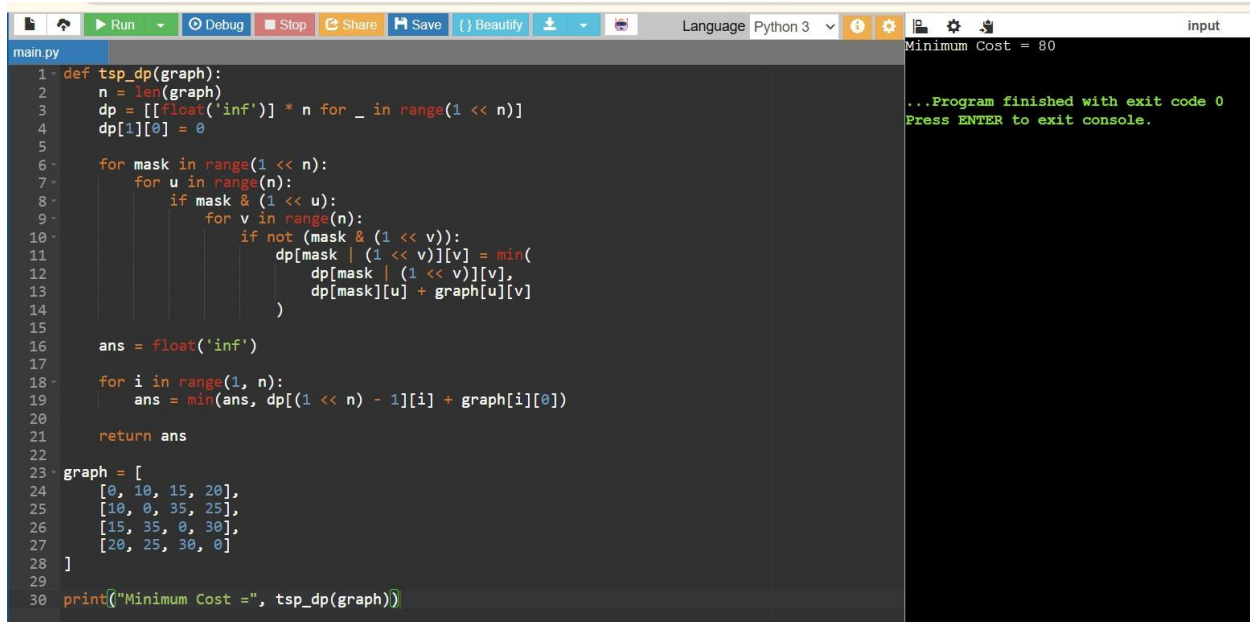
graph = [
    [0,10,15,20],
    [10,0,35,25],
    [15,35,0,30],
    [20,25,30,0]
]

print("Minimum Cost =", tsp_dp(graph))

```

Output

Minimum Cost = 80



```
1 def tsp_dp(graph):
2     n = len(graph)
3     dp = [[float('inf')] * n for _ in range(1 << n)]
4     dp[1][0] = 0
5
6     for mask in range(1 << n):
7         for u in range(n):
8             if mask & (1 << u):
9                 for v in range(n):
10                    if not (mask & (1 << v)):
11                        dp[mask | (1 << v)][v] = min(
12                            dp[mask | (1 << v)][v],
13                            dp[mask][u] + graph[u][v]
14                        )
15
16     ans = float('inf')
17
18     for i in range(1, n):
19         ans = min(ans, dp[(1 << n) - 1][i] + graph[i][0])
20
21     return ans
22
23 graph = [
24     [0, 10, 15, 20],
25     [10, 0, 35, 25],
26     [15, 35, 0, 30],
27     [20, 25, 30, 0]
28 ]
29
30 print("Minimum Cost =", tsp_dp(graph))
```

Minimum Cost = 80

...Program finished with exit code 0
Press ENTER to exit console.

9. Optimal Binary Search Tree (OBST)

Program

```
def optimal_bst(freq):
    n = len(freq)
    cost = [[0]*n for _ in range(n)]
    for i in range(n):
        cost[i][i] = freq[i]
        for L in range(2, n+1):
            for i in range(n-L+1):
                j = i+L-1
                cost[i][j] = float('inf')
                total = sum(freq[i:j+1])
                for r in range(i, j+1):
                    left = cost[i][r-1] if r > i else 0
                    right = cost[r+1][j] if r < j else 0
                    cost[i][j] = min(cost[i][j],
                                     left + right + total)
    return cost[0][n-1]

print("Optimal Cost =", optimal_bst([34,8,50]))
```

Output

Optimal Cost = 142


```

1 def optimal_bst(freq):
2     n = len(freq)
3     cost = [[0] * n for _ in range(n)]
4
5     for i in range(n):
6         cost[i][i] = freq[i]
7
8     for L in range(2, n + 1):
9         for i in range(n - L + 1):
10             j = i + L - 1
11             cost[i][j] = float('inf')
12             total = sum(freq[i:j + 1])
13
14             for r in range(i, j + 1):
15                 left = cost[i][r - 1] if r > i else 0
16                 right = cost[r + 1][j] if r < j else 0
17                 cost[i][j] = min(cost[i][j], left + right + total)
18
19     return cost[0][n - 1]
20
21 print("Optimal Cost =", optimal_bst([34, 8, 50]))

```

Optimal Cost = 142

...Program finished with exit code 0
Press ENTER to exit console.

10. Matrix Chain Multiplication

Program

```

def matrix_chain(arr):
    n = len(arr)
    dp = [[0]*n for _ in range(n)]
    for L in range(2, n):
        for i in range(1, n-L+1):
            j = i+L-1
            dp[i][j] = float('inf')
            for k in range(i, j):
                q = dp[i][k] + dp[k+1][j] + arr[i-1]*arr[k]*arr[j]
                dp[i][j] = min(dp[i][j], q)
            return dp[1][n-1]

```

```

print("Minimum Multiplications =", matrix_chain([10,20,30,40,30]))

```

Output

Minimum Multiplications = 30000

```

1 def matrix_chain(arr):
2     n = len(arr)
3     dp = [[0] * n for _ in range(n)]
4
5     for L in range(2, n):
6         for i in range(1, n - L + 1):
7             j = i + L - 1
8             dp[i][j] = float('inf')
9
10            for k in range(i, j):
11                q = dp[i][k] + dp[k + 1][j] + arr[i - 1] * arr[k] * arr[j]
12                dp[i][j] = min(dp[i][j], q)
13
14            return dp[1][n - 1]
15
16 print("Minimum Multiplications =", matrix_chain([10, 20, 30, 40, 30]))

```

Minimum Multiplications = 30000

...Program finished with exit code 0
Press ENTER to exit console.

11. Coin Change (Minimum Coins)

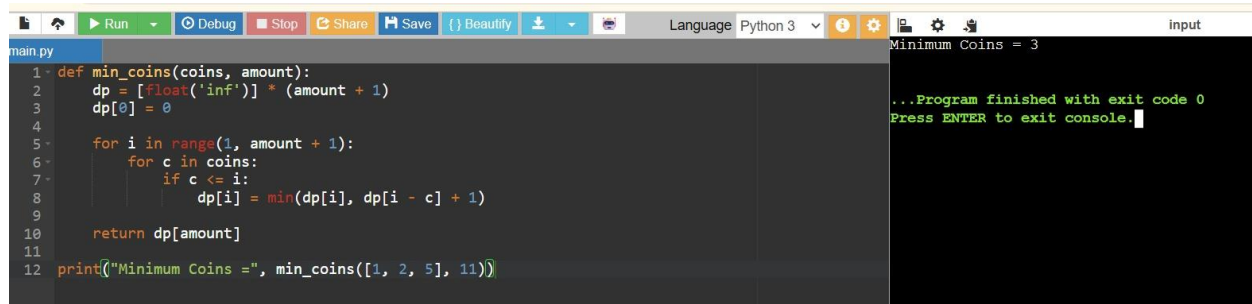
Program

```
def min_coins(coins, amount):
    dp = [float('inf')] * (amount+1)
    dp[0] = 0
    for i in range(1, amount+1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i-c]+1)
    return dp[amount]

print("Minimum Coins =", min_coins([1,2,5], 11))
```

Output

Minimum Coins = 3

A screenshot of a code editor interface. The editor has a top toolbar with icons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3. The code in the editor is the same as the one shown in the previous block. The output on the right shows "Minimum Coins = 3" and a message: "...Program finished with exit code 0 Press ENTER to exit console."

12. Rod Cutting Problem

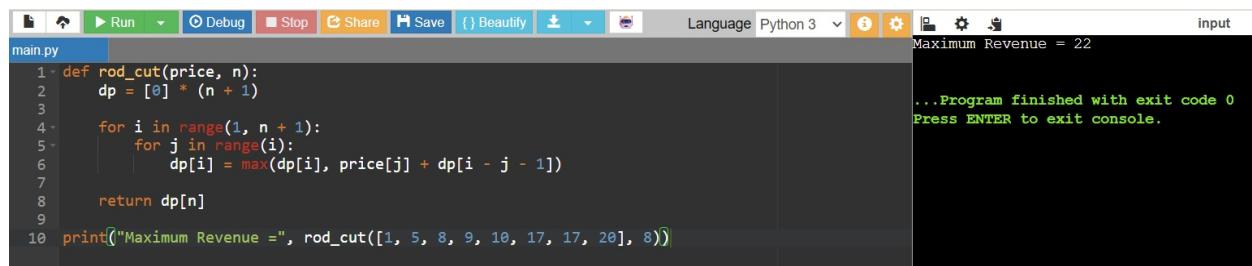
Program

```
def rod_cut(price, n):
    dp = [0]*(n+1)
    for i in range(1, n+1):
        for j in range(i):
            dp[i] = max(dp[i],
                        price[j] + dp[i-j-1])
    return dp[n]

print("Maximum Revenue =", rod_cut([1,5,8,9,10,17,17,20], 8))
```

Output

Maximum Revenue = 22

A screenshot of a code editor interface. The editor has a top toolbar with icons for Run, Debug, Stop, Share, Save, and Beautify. The language is set to Python 3. The code in the editor is the same as the one shown in the previous block. The output on the right shows "Maximum Revenue = 22" and a message: "...Program finished with exit code 0 Press ENTER to exit console."

13. Minimum Cost Path

Program

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0]*n for _ in range(m)]
    dp[0][0] = cost[0][0]
    for i in range(1,m):
        dp[i][0] = dp[i-1][0] + cost[i][0]
    for j in range(1,n)
        dp[0][j] = dp[0][j-1] + cost[0][j]
    for i in range(1,m):
        for j in range(1,n):
            dp[i][j] = cost[i][j] + min(dp[i-1][j],
                                         dp[i][j-1])

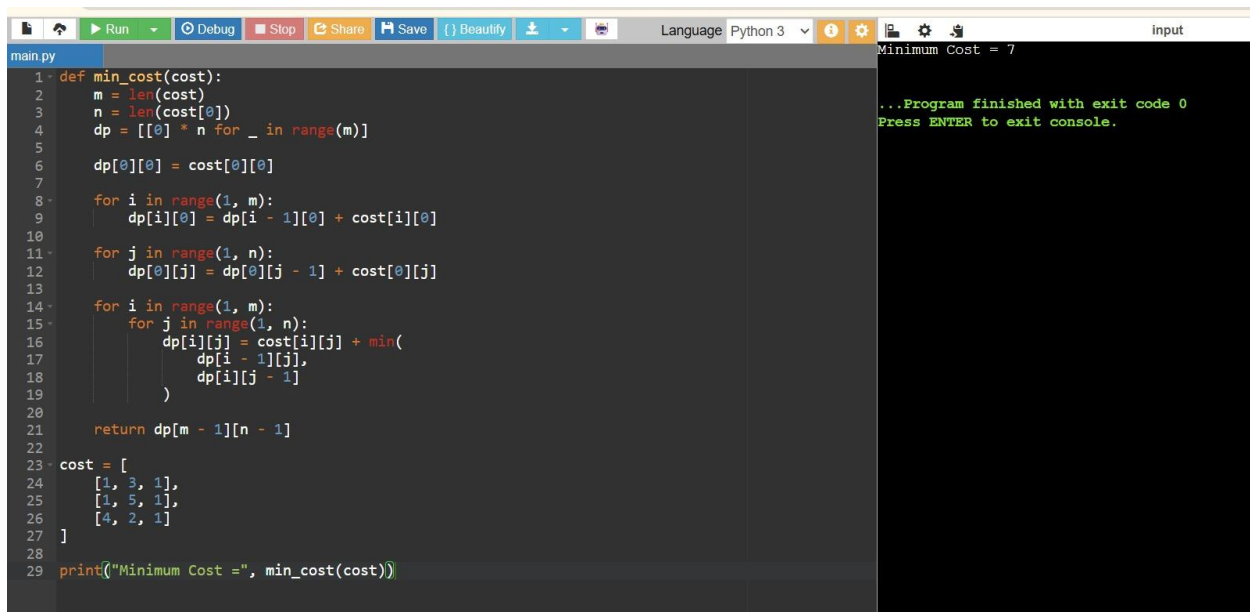
    return dp[m-1][n-1]

cost = [
    [1,3,1],
    [1,5,1],
    [4,2,1]
]

print("Minimum Cost =", min_cost(cost))
```

Output

Minimum Cost = 7



The screenshot shows a code editor with a dark theme. The left pane contains the Python code for the Minimum Cost Path problem. The code defines a function `min_cost` that uses dynamic programming to find the minimum cost path from the top-left to the bottom-right of a grid. It initializes a `dp` table and fills it with the minimum cost to reach each cell. The `cost` matrix is defined as `[[1, 3, 1], [1, 5, 1], [4, 2, 1]]`. The main execution calls `min_cost(cost)` and prints the result. The right pane shows the output: `Minimum Cost = 7`. Below the output, it says `...Program finished with exit code 0` and `Press ENTER to exit console.`

```
1 def min_cost(cost):
2     m = len(cost)
3     n = len(cost[0])
4     dp = [[0]*n for _ in range(m)]
5
6     dp[0][0] = cost[0][0]
7
8     for i in range(1, m):
9         dp[i][0] = dp[i-1][0] + cost[i][0]
10
11    for j in range(1, n):
12        dp[0][j] = dp[0][j-1] + cost[0][j]
13
14    for i in range(1, m):
15        for j in range(1, n):
16            dp[i][j] = cost[i][j] + min(
17                dp[i-1][j],
18                dp[i][j-1]
19            )
20
21    return dp[m-1][n-1]
22
23 cost = [
24     [1, 3, 1],
25     [1, 5, 1],
26     [4, 2, 1]
27 ]
28
29 print("Minimum Cost =", min_cost(cost))
```

Minimum Cost = 7

...Program finished with exit code 0
Press ENTER to exit console.